Object Oriented Programming

15-3-26

Aim – Deign a system using classes for vehicles, rental agencies, and rental transactions. Implement methods to handle vehicle availability, rental periods, pricing, and customer bookings.

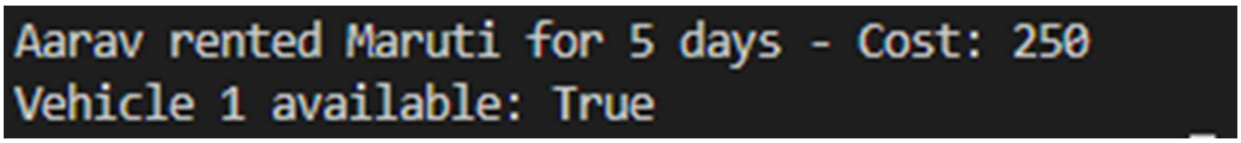
Theory –

Object-Oriented Programming in Python organizes code using classes and objects. A class defines attributes and methods, while objects are instances of classes. Inheritance allows new classes to reuse properties from parent classes. Method overriding enables subclasses to modify behavior. This approach improves modularity, reusability, and structure of programs.

A1.

```
from datetime import datetime, timedelta


class Vehicle:
    def __init__(self, id, name, daily_rate):
        self.id = id
        self.name = name
        self.daily_rate = daily_rate
        self.available = True


class RentalAgency:
    def __init__(self, name):
        self.name = name
        self.vehicles = []
        self.transactions = []


    def add_vehicle(self, vehicle):
        self.vehicles.append(vehicle)


    def book_vehicle(self, vehicle_id, customer, days):
        vehicle = next((v for v in self.vehicles if v.id == vehicle_id), None)
        if not vehicle or not vehicle.available:
            return None
```

```
        cost = vehicle.daily_rate * days
        rental = RentalTransaction(vehicle, customer, days, cost)
        vehicle.available = False
        self.transactions.append(rental)
        return rental

    def return_vehicle(self, vehicle_id):
        vehicle = next((v for v in self.vehicles if v.id == vehicle_id), None)
        if vehicle:
            vehicle.available = True

class RentalTransaction:
    def __init__(self, vehicle, customer, days, cost):
        self.vehicle = vehicle
        self.customer = customer
        self.days = days
        self.cost = cost
        self.start_date = datetime.now()
        self.end_date = self.start_date + timedelta(days=days)

    def __str__(self):
        return f"{self.customer} rented {self.vehicle.name} for {self.days} days - Cost: {self.cost}"

agency = RentalAgency("QuickRent")
agency.add_vehicle(Vehicle(1, "Maruti", 50))
agency.add_vehicle(Vehicle(2, "Honda", 60))

rental = agency.book_vehicle(1, "Aarav", 5)
print(rental)

agency.return_vehicle(1)
print(f"Vehicle 1 available: {agency.vehicles[0].available}")
```



Conclusion

OOP is used to encapsulate various functions and provide data abstraction.